MINISTRY OF EDUCATION AND TRAINING

TRƯỜNG ĐẠI HỌC FPT

FPT UNIVERSITY

Capstone Project Document

THE LAST SIGNAL: A 2D Top-Down Asymmetric Multiplayer Horror Game with Dedicated Game Servers and an Online Service Platform.



Group Name: GFA26SE54	
Group Members	Cao Thành Danh – SE194497 – Team Leader Nguyễn Công Thiên Ân – SE194544 – Member Hồ Trọng – SE194602 – Member Phan Doãn Thuận – SE194684 – Member

Supervisor	Nguyễn Thế Hoàng
Ext Supervisor	N/A
Capstone Project code	FA26SE / GFA26SE54

- Ho Chi Minh City, 03/2027 -

Table of Contents

Acknowledgement 4
Definition and Acronyms 4
I. Project Introduction 5
1. Overview 5
2. Product Background 5
3. Existing Systems 5
4. Business Opportunity 5
5. Software Product Vision 5
6. Project Scope & Limitations 5
II. Project Management Plan 6
1. Overview 6
2. Management Approach 6
3. Master Schedule 6
4. Project Organization 6
5. Project Communication 6
6. Configuration Management 6
III. Software Requirement Specification 7
1. Overall Description 7
2. User Requirements 7
3. Functional Requirements 7
4. Non-Functional Requirements 7
5. Other Requirements 7
IV. Software Design Description 8

1. Overall Description 8
2. System Architecture Design 8
3. System Detailed Design 8
4. Class Specification 8
5. Data & Database Design 8
V. Software Testing Documentation 9
1. Overall Description 9
2. Test Plan 9
3. Test Cases 9
4. Test Reports 9
VI. Release Package & User Guides 10
1. Deliverable Package 10
2. Installation Guides 10
3. User Manual 10
VII. Appendix 10
1. Glossary [Optional] 10
2. References [Optional] 10
3. Others [Optional] 10

Acknowledgement

We would like to express our sincere gratitude to our supervisor, **Mr. Nguyen The Hoang**, for his continuous guidance, technical feedback, and patience throughout the development of *THE LAST SIGNAL*.

We would also like to extend our heartfelt thanks to **FPT University Ho Chi Minh City**, our lecturers, our families, our friends, and our teammates for their encouragement, support, and valuable contributions throughout this journey. Their motivation has been an important source of inspiration, enabling us to remain committed and to successfully complete this capstone project.

Definition and Acronyms

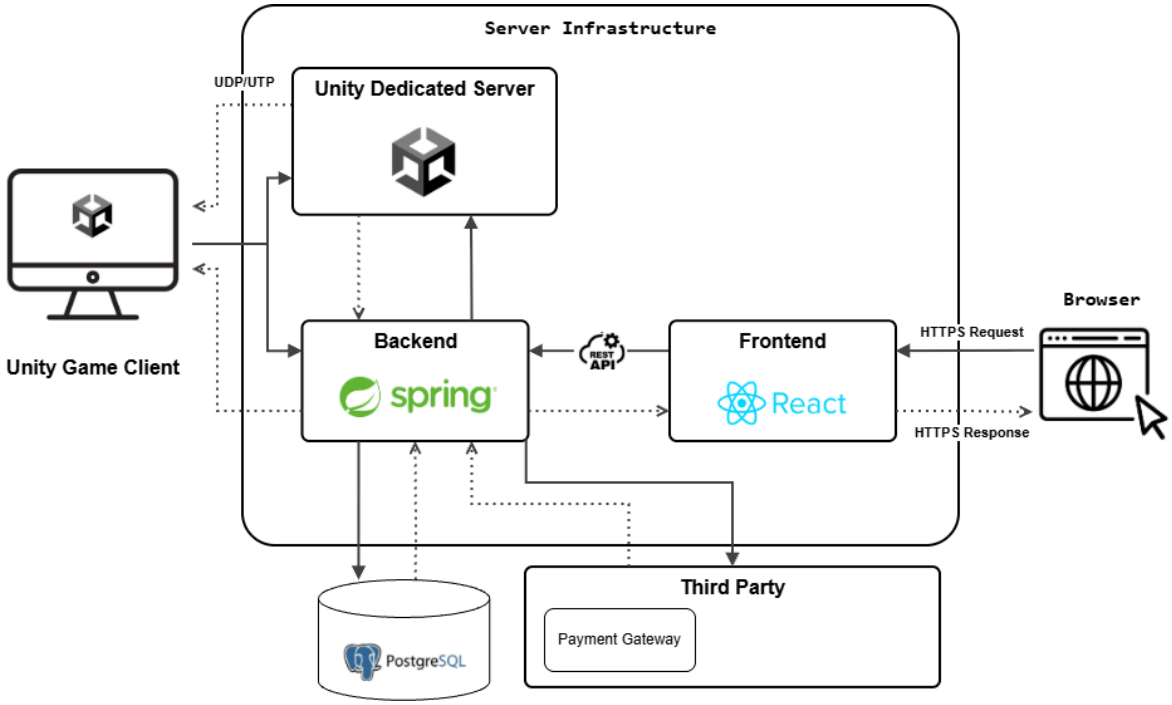
The table below lists the acronyms and domain-specific terms used throughout this document.

Acronym	Definition
NGO	Unity Netcode for GameObjects — the multiplayer networking framework used
UTP	Unity Transport Package
UGS	Unity Gaming Services (Relay, Authentication)
CCU	Concurrent Users (Relay service billing metric)
FSM	Finite State Machine
FOV	Field of View — raycast-based enemy vision detection
NavMesh	Navigation Mesh used for AI pathfinding
URP	Universal Render Pipeline
RPC	Remote Procedure Call
LAN	Local Area Network
HUD	Heads-Up Display
FR	Functional Requirement
NFR	Non-Functional Requirement
MVP	Minimum Viable Product
SRS	Software Requirement Specification
SAD	Software Architecture Document
GDD	Game Design Document
UC	Use Case
API	Application Programming Interface
GUI	Graphical User Interface

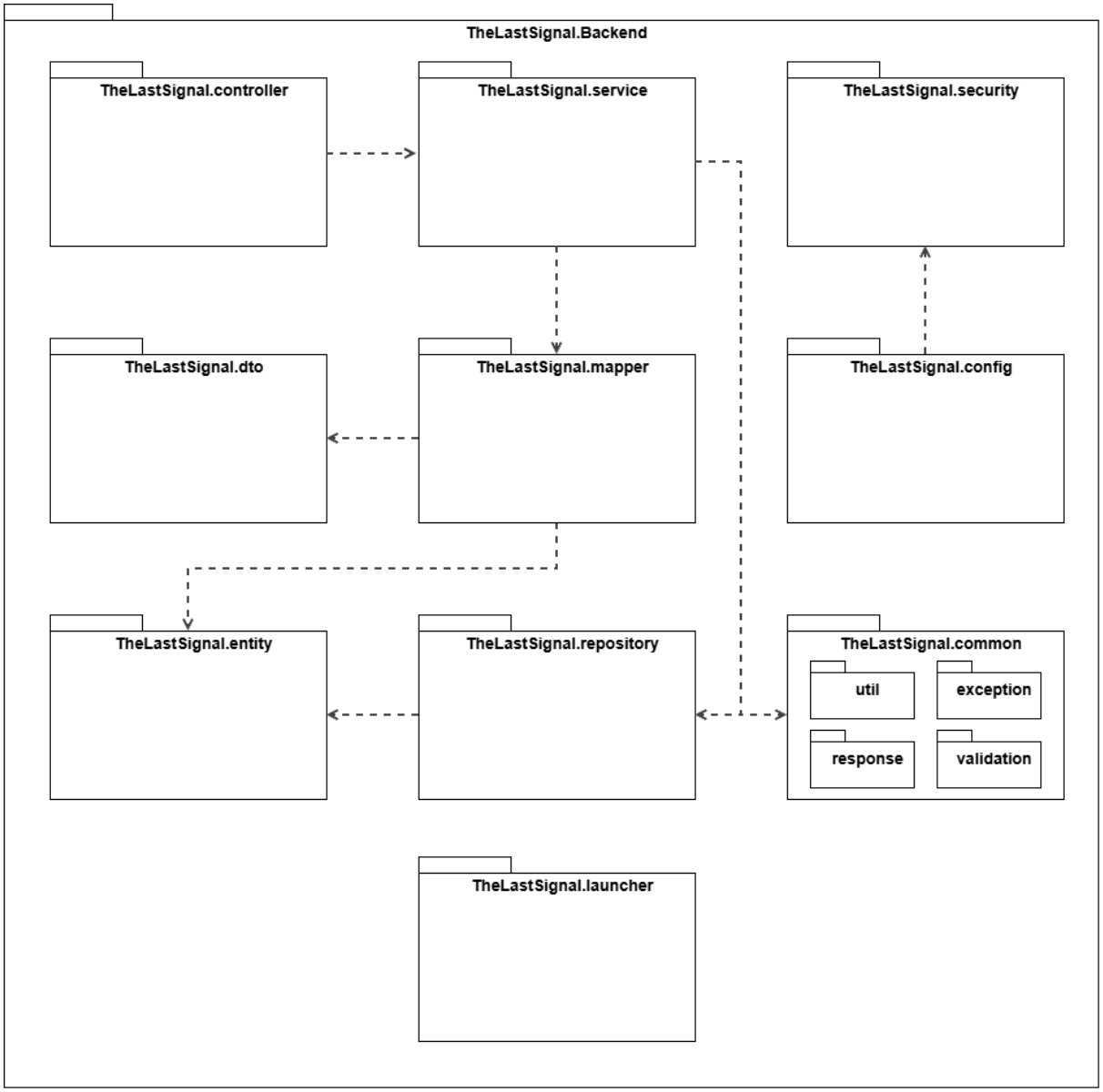
IV. Software Design Description

1. System Design

1.1 System Architecture



1.2 Package Diagram



2. Database Design

The persistence layer of THE LAST SIGNAL uses a relational database schema managed via PostgreSQL 16 and versioned through Flyway migration scripts. The database maintains transactional integrity, player progression, match auditing, store purchases, player bug reports, and dynamic gameplay balance configurations. Transient lobby room coordination is maintained in-memory within the Spring Boot application (via ConcurrentHashMap) for optimal responsiveness, whereas durable domain entities are fully normalized into 9 connected relational tables.

[SDD-DIAG-03: Complete Database Entity-Relationship Diagram (9 Tables)]

Table Descriptions

No	Table	Description

01	users	Stores player credentials, account role, activation state (enabled: true = ACTIVE, false = DEACTIVE), and registration metadata. - Primary keys: id (UUID) - Foreign keys: None
02	player_stats	Tracks player career progression, victory metrics, escapes, repairs, revives, and incapacitations (1:1 with users: matches_played, wins, losses, aborted_matches, escapes, machines_repaired, teammates_revived, times_incapacitated). - Primary keys: user_id (UUID) - Foreign keys: user_id references users(id)
03	game_sessions	Records dedicated game server match sessions, lifecycle states, duration, idempotency key, server secret hash, and authoritative match outcomes. - Primary keys: id (UUID) - Foreign keys: None
04	match_players	Junction entity storing per-player participation details, slot index, character role, join tokens, per-match contributions (machines_repaired, teammates_revived, incapacitated), and final individual outcome status. - Primary keys: id (UUID) - Foreign keys: match_id references game_sessions(id), user_id references users(id)
05	skins	Cosmetic skin catalog defining available character skins, pricing, and visual assets. - Primary keys: id (UUID) - Foreign keys: None
06	user_skins	Player inventory maintaining owned cosmetic skins and active equipped status. - Primary keys: id (UUID) - Foreign keys: user_id references users(id), skin_id references skins(id)
07	payment_transactions	Financial transaction ledger for cosmetic purchases and payment gateway webhooks. - Primary keys: id (UUID) - Foreign keys: user_id references users(id), skin_id references skins(id)
08	bug_reports	Player-submitted bug and gameplay issue reports triaged via the Web Admin Portal (category, title, description, client_log, status, admin_note). - Primary keys: id (UUID)

		- Foreign keys: reporter_id references users(id), resolved_by references users(id)
09	balance_configs	Versioned gameplay balance parameters managed dynamically by administrators (required_repairs, sanity_drain_rate, sanity_low_threshold, survivor_base_stamina, shade_light_slow_multiplier, downed_crawl_speed_ratio, revive_health_ratio). - Primary keys: id (UUID) - Foreign keys: updated_by references users(id)

3. Detailed Design

This section provides the detailed software design for all server-side platform and web features of THE LAST SIGNAL, covering class structures and sequence diagrams for core operational scenarios. Following the project scoping guidelines, client-side real-time Unity gameplay simulation (such as player physics movement, lighting raycasts, and local audio) is excluded from this section as it is maintained independently within the Unity client engine.

3.1 Account & Profile Management

The Account & Profile Management feature encapsulates player identity authentication, account registration, secure token issuance, and career statistical aggregation. Security credentials are encrypted using BCrypt, and sessions are authorized via signed JWT access tokens containing user claims.

3.1.1 Class Structure

[SDD-DIAG-04: Account & Profile Management Class Diagram]

The class structure is anchored around AuthController and ProfileController in the presentation layer, delegating to AuthService and ProfileService. Persistence is managed via UserRepository and PlayerStatsRepository accessing the User and PlayerStats JPA entities.

3.1.2 Sequence Flows

3.1.2.1 Player Registration Flow

[SDD-DIAG-05: Player Registration Sequence Diagram]

Step 1: Player inputs registration credentials (username, email, password) in the client interface.

Step 2: Game Client sends HTTP POST /api/v1/auth/register with RegisterRequest payload.

Step 3: AuthController validates request parameters using Jakarta Bean Validation.

Step 4: AuthService verifies that username and email do not already exist via UserRepository.

Step 5: AuthService hashes the raw password using BCryptPasswordEncoder.

Step 6: AuthService persists the new User entity and initializes a corresponding PlayerStats record with zeroed counters.

Step 7: AuthService invokes JwtService to generate an HMAC-SHA256 signed JWT access token.

Step 8: AuthController returns HTTP 201 Created with AuthResponse containing token and player profile details.

3.1.2.2 Player Login & Authentication Flow

[SDD-DIAG-06: Player Login Sequence Diagram]

Step 1: Player provides login credentials (username, password) on the login screen.

Step 2: Game Client submits HTTP POST /api/v1/auth/login containing LoginRequest.

Step 3: AuthController forwards credentials to AuthService.

Step 4: AuthService retrieves the User entity by username from UserRepository.

Step 5: AuthService compares password hash via BCryptPasswordEncoder; throws UNAUTHORIZED if invalid.

Step 6: AuthService checks account enabled flag (true = ACTIVE, false = DEACTIVE); throws FORBIDDEN if account is DEACTIVE.

Step 7: JwtService issues an authenticated JWT access token containing subject ID, username, and role.

Step 8: AuthController returns HTTP 200 OK enclosing the JWT token and user profile.

3.1.2.3 Profile & Statistics Retrieval Flow

[SDD-DIAG-07: Get Career Profile & Statistics Sequence Diagram]

Step 1: Authenticated Player opens the Profile / Career Statistics screen.

Step 2: Game Client issues HTTP GET /api/v1/profile with Bearer JWT token in Authorization header.

Step 3: JwtAuthenticationFilter validates JWT token signature and populates Spring SecurityContext.

Step 4: ProfileController extracts authenticated User ID and calls ProfileService.getProfile(userId).

Step 5: ProfileService queries UserRepository and PlayerStatsRepository for career records.

Step 6: ProfileService computes win rate percentage and total matches completed.

Step 7: ProfileController returns HTTP 200 OK with PlayerProfileResponse.

3.2 Room & Lobby Management

The Room & Lobby Management subsystem handles ephemeral multiplayer match assembly, room creation, 6-character room code generation, player slot assignment, readiness toggling, and match start authorization. Lobby states are maintained in-memory within RoomService using thread-safe ConcurrentHashMap structures for high throughput and zero persistence latency.

3.2.1 Class Structure

[SDD-DIAG-08: Room & Lobby Management Class Diagram]

RoomController exposes REST endpoints for room lifecycle actions, invoking RoomService. In-memory models Room and Player track room slots, host authority, ready statuses, and assigned player roles.

3.2.2 Sequence Flows

3.2.2.1 Room Creation Flow

[SDD-DIAG-09: Create Room Sequence Diagram]

Step 1: Host player selects 'Create Room' from the main menu.

Step 2: Game Client sends HTTP POST /api/v1/rooms with Bearer JWT token.

Step 3: RoomController delegates to RoomService.createRoom(hostUserId).

Step 4: RoomService generates a cryptographically unique 6-character alphanumeric room code matching ^[A-Z0-9]{6}$.

Step 5: RoomService instantiates an in-memory Room object, sets maxPlayers to 4, and registers the host in slot 0 as host.

Step 6: RoomService stores the room instance in the concurrent room registry map.

Step 7: RoomController returns HTTP 201 Created with RoomDto containing room code, host ID, and initial slot list.

3.2.2.2 Join Room Flow

[SDD-DIAG-10: Join Room Sequence Diagram]

Step 1: Client enters a 6-character room code and clicks 'Join Room'.

Step 2: Game Client sends HTTP POST /api/v1/rooms/{code}/join with Bearer JWT token.

Step 3: RoomController validates room code syntax and invokes RoomService.joinRoom(code, userId).

Step 4: RoomService checks if the target room exists and is currently in WAITING status.

Step 5: RoomService verifies that current player count is less than 4 and user is not already in another room.

Step 6: RoomService assigns the player to the next vacant slot (index 1-3) with isReady = false.

Step 7: RoomController returns HTTP 200 OK with updated RoomDto.

3.2.2.3 Toggle Ready & Match Initiation Flow

[SDD-DIAG-11: Toggle Ready & Start Match Sequence Diagram]

Step 1: Connected players click 'Ready'; host clicks 'Start Match' once all players are ready.

Step 2: Game Client sends HTTP POST /api/v1/rooms/{code}/ready or /api/v1/rooms/{code}/start.

Step 3: RoomService updates player readiness state in the in-memory room structure.

Step 4: Upon Start request, RoomService verifies that the caller is the Room Host and all 4 players are marked ready.

Step 5: RoomService assigns game roles: 1 Shade and 3 Survivors (Engineer, Drone Operator, Logistic).

Step 6: RoomService coordinates with GameServerService to allocate an available Dedicated Server instance.

Step 7: RoomService transitions room state to STARTING and generates secure join tokens for each participant.

Step 8: RoomController returns HTTP 200 OK with server connection details and player join ticket.

3.3 Match Session & Result Processing

The Match Session & Result Processing feature governs authoritative match execution on dedicated game servers, single-use join ticket validation, server heartbeat monitoring, and idempotent match result ingestion. To prevent race conditions and duplicate career stat increments, the backend enforces pessimistic database locking and unique idempotency constraints.

3.3.1 Class Structure

[SDD-DIAG-12: Match Session & Result Processing Class Diagram]

MatchServerController and SessionController expose dedicated server communication interfaces, delegating to MatchService and SessionService. Entities GameSession and MatchPlayer persist match history, outcome details, and token hashes.

3.3.2 Sequence Flows

3.3.2.1 Dedicated Server Join-Token Validation Flow

[SDD-DIAG-13: Dedicated Server Join-Token Validation Sequence Diagram]

Step 1: Game Client connects to Unity Dedicated Server via UDP Transport supplying its join-token string.

Step 2: Dedicated Server sends HTTP POST /api/v1/server/matches/validate-token with X-Server-Secret header and token.

Step 3: MatchServerController verifies X-Server-Secret against server_secret_hash.

Step 4: MatchService hashes the incoming token using SHA-256 and queries MatchPlayer by token hash.

Step 5: MatchService verifies that token has not expired and has not been used previously.

Step 6: MatchService marks joined_at timestamp on the MatchPlayer record.

Step 7: MatchServerController returns HTTP 200 OK with ValidateTokenResponse containing player userId, slotIndex, and assigned role.

3.3.2.2 Dedicated Server Heartbeat Flow

[SDD-DIAG-14: Server Heartbeat Monitoring Sequence Diagram]

Step 1: Running Dedicated Server periodically sends HTTP POST /api/v1/server/matches/{matchId}/heartbeat every 30 seconds.

Step 2: MatchServerController validates server authentication secret.

Step 3: MatchService updates last_heartbeat_at timestamp on the corresponding GameSession record.

Step 4: MatchServerController returns HTTP 200 OK acknowledging active session state.

3.3.2.3 Authoritative Match Result Ingestion Flow

[SDD-DIAG-15: Match Result Ingestion & Statistics Update Sequence Diagram]

Step 1: Match concludes on Dedicated Server (escape threshold reached or all survivors eliminated).

Step 2: Dedicated Server sends HTTP POST /api/v1/server/matches/{matchId}/results with MatchResultRequest and resultIdempotencyKey.

Step 3: MatchServerController authenticates dedicated server secret header.

Step 4: MatchService checks result_idempotency_key on GameSession; if already processed, immediately returns cached 200 OK.

Step 5: MatchService acquires pessimistic row lock (PESSIMISTIC_WRITE) on GameSession.

Step 6: MatchService records final outcome, duration_seconds, ended_reason, and updates each MatchPlayer final_status.

Step 7: MatchService updates PlayerStats records for all participants (incrementing matches_played, wins, losses, escapes, machines_repaired, teammates_revived, and times_incapacitated).

Step 8: MatchService commits transaction and returns HTTP 200 OK confirming result persisted.

3.4 Cosmetic Skin Store & Payment Processing

The Cosmetic Skin Store & Payment Processing feature enables players to browse character skins, initiate purchases, and complete transactions via third-party payment gateways (e.g., VNPay sandbox). Webhooks from payment gateways are cryptographically verified to ensure tamper-proof inventory provisioning.

3.4.1 Class Structure

[SDD-DIAG-16: Cosmetic Skin Store & Payment Processing Class Diagram]

StoreController and PaymentWebhookController interact with SkinService and PaymentService. Entities Skin, UserSkin, and PaymentTransaction maintain catalog pricing, user inventories, and financial audit logs.

3.4.2 Sequence Flows

3.4.2.1 Skin Purchase Initiation Flow

[SDD-DIAG-17: Cosmetic Purchase & Payment Gateway Initiation Sequence Diagram]

Step 1: Player browses store catalog and selects 'Purchase Skin'.

Step 2: Game Client sends HTTP POST /api/v1/store/skins/{skinId}/purchase with Bearer JWT token.

Step 3: StoreController invokes PaymentService.initiatePurchase(userId, skinId).

Step 4: PaymentService verifies skin is active and player does not already own the skin in UserSkin repository.

Step 5: PaymentService generates unique transactionCode and creates PaymentTransaction record in PENDING state.

Step 6: PaymentService calls external Payment Gateway API to generate secure payment checkout URL with digital signature.

Step 7: StoreController returns HTTP 200 OK with checkout URL and transaction code.

3.4.2.2 Payment Gateway Webhook Handling Flow

[SDD-DIAG-18: Payment Gateway Webhook Callback Sequence Diagram]

Step 1: Payment gateway redirects player and sends asynchronous HTTP POST /api/v1/webhooks/payment/callback.

Step 2: PaymentWebhookController receives callback payload and gateway signature header.

Step 3: PaymentService verifies cryptographic HMAC signature using shared payment gateway secret.

Step 4: PaymentService retrieves PaymentTransaction by transactionCode.

Step 5: If payment status is SUCCESS, PaymentService updates transaction status to SUCCESS and records paid_at timestamp.

Step 6: PaymentService inserts a new UserSkin record into player inventory.

Step 7: PaymentWebhookController returns HTTP 200 OK acknowledgement to Payment Gateway.

3.5 Bug & Issue Reporting

The Bug & Issue Reporting feature allows players to submit issue tickets and diagnostic logs directly from the game client. Administrators triage, investigate, and resolve submitted reports through the Web Admin Portal.

3.5.1 Class Structure

[SDD-DIAG-19: Bug & Issue Reporting Class Diagram]

BugReportController handles client submissions and admin management endpoints, delegating to BugReportService. BugReport entity stores issue metadata, crash logs, and admin resolution comments.

3.5.2 Sequence Flows

3.5.2.1 Bug Report Submission Flow

[SDD-DIAG-20: Bug Report Submission Flow Sequence Diagram]

Step 1: Player encounters an in-game defect, opens 'Report Bug' modal, and enters details.

Step 2: Game Client collects optional local log snippets and sends HTTP POST /api/v1/reports with CreateBugReportRequest.

Step 3: BugReportController validates input fields and calls BugReportService.submitReport(userId, request).

Step 4: BugReportService creates BugReport entity with status = OPEN and persists it to database.

Step 5: BugReportController returns HTTP 201 Created with created report reference ID.

3.6 Web Admin Portal Management

The Web Admin Portal provides administrative control over player accounts, match session auditing, bug report triage, and dynamic gameplay balance configurations. Access is strictly restricted to users holding ROLE_ADMIN authority.

3.6.1 Class Structure

[SDD-DIAG-21: Web Admin Portal Class Diagram]

AdminController coordinates AdminUserService, AdminSessionService, AdminBugReportService, and AdminBalanceConfigService. BalanceConfig entity stores live game balance tunables.

3.6.2 Sequence Flows

3.6.2.1 Balance Configuration Update Flow

[SDD-DIAG-22: Admin Balance Configuration Update Sequence Diagram]

Step 1: Admin navigates to Balance Config screen on React Admin Portal and adjusts gameplay parameters.

Step 2: Admin Portal issues HTTP PUT /api/v1/admin/balance-configs/{configId} with UpdateBalanceConfigRequest and Admin JWT.

Step 3: JwtAuthenticationFilter verifies caller holds ROLE_ADMIN authority; throws 403 Forbidden otherwise.

Step 4: AdminBalanceConfigService retrieves existing BalanceConfig entity.

Step 5: AdminBalanceConfigService updates balance values (repair requirements, sanity drain rate, speed multipliers) and sets updated_by.

Step 6: AdminBalanceConfigService saves entity and invalidates active balance cache.

Step 7: AdminController returns HTTP 200 OK with updated configuration details.

3.6.2.2 Player Account Suspension Flow

[SDD-DIAG-23: Admin Player Account Status Update (ACTIVE / DEACTIVE) Sequence Diagram]

Step 1: Admin selects a player account on Admin Portal and clicks 'Deactivate Account' (or 'Activate Account').

Step 2: Admin Portal sends HTTP PATCH /api/v1/admin/users/{userId}/status with enabled = false.

Step 3: AdminUserService updates User entity enabled flag to false in UserRepository.

Step 4: AdminUserService terminates any active lobby sessions associated with the user.

Step 5: AdminController returns HTTP 200 OK confirming account status updated.